Name: M. Sivateja

Hall Ticket: 2303A521445

Batch: 41

Task 1: Stack Implementation for Undo Operations (LIFO)

Prompt:

Generate a Python Stack class with push, pop, peek, is_empty methods.

Add docstrings and comments. Also give example for undo operations.

Compare list vs deque for stack.

Code:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, action):
        self.stack.append(action)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0

    def display(self):
        print(f"\nCurrent Stack: {self.stack}")


def interactive_stack():
    s = Stack()

    while True:
        print("\n--- Interactive Stack Operations (List-based) ---")
        s.display()
        print("1. Push (add action)")
        print("2. Pop (undo last action)")
        print("3. Peek (view last action)")
        print("4. Check if Empty")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")
```

```
        if choice == '1':
            action = input("Enter action to push: ")
            s.push(action)
            print(f"'{action}' pushed to stack.")

        elif choice == '2':
            popped = s.pop()
            if popped:
                print(f"'{popped}' popped (undone).")
            else:
                print("Stack is empty.")

        elif choice == '3':
            top = s.peek()
            if top:
                print(f"Top element: '{top}'")
            else:
                print("Stack is empty.")

        elif choice == '4':
            if s.is_empty():
                print("Stack is empty.")
            else:
                print("Stack is not empty.")

        elif choice == '5':
            print("Exiting interactive stack operations.")
            break

        else:
            print("Invalid choice. Try again.")


# Run program
interactive_stack()
```

```
    --- Interactive Stack Operations (List-based) ---
...
    Current Stack: []
    1. Push (add action)
    2. Pop (undo last action)
    3. Peek (view last action)
    4. Check if Empty
    5. Exit
    Enter your choice (1-5): 1
    Enter action to push: 2
    '2' pushed to stack.

    --- Interactive Stack Operations (List-based) ---

    Current Stack: ['2']
    1. Push (add action)
    2. Pop (undo last action)
    3. Peek (view last action)
    4. Check if Empty
    5. Exit
    Enter your choice (1-5): 3
    Top element: '2'

    --- Interactive Stack Operations (List-based) ---

    Current Stack: ['2']
    1. Push (add action)
    2. Pop (undo last action)
    3. Peek (view last action)
    4. Check if Empty
    5. Exit
    Enter your choice (1-5): 4
    Stack is not empty.

    --- Interactive Stack Operations (List-based) ---

    Current Stack: ['2']
```

```
--- Interactive Stack Operations (List-based) ---
...
Current Stack: ['2']
1. Push (add action)
2. Pop (undo last action)
3. Peek (view last action)
4. Check if Empty
5. Exit
Enter your choice (1-5): 4
Stack is not empty.

--- Interactive Stack Operations (List-based) ---

Current Stack: ['2']
1. Push (add action)
2. Pop (undo last action)
3. Peek (view last action)
4. Check if Empty
5. Exit
Enter your choice (1-5): 5
Exiting interactive stack operations.
```

+ Code + Text

Explanation:

A stack follows **LIFO (Last In, First Out)**, making it ideal for undo operations where the most recent action is reversed first.
The program uses a Python list with methods like push, pop, peek, and is_empty, all working in **O(1) time**.
An interactive menu lets users perform operations dynamically while handling edge cases like empty stack

Task 2 Queue for Customer Service Requests (FIFO)

Prompt:

Create a Queue class in Python using list with enqueue, dequeue, is_empty methods.

Then implement optimized queue using collections.deque.

Add comments, sample test cases, and explain performance difference.

Code:

```python
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return None

    def is_empty(self):
        return len(self.queue) == 0
```

```python
    def display(self):
        print(f"\nCurrent Queue: {self.queue}")


def interactive_queue():
    q = Queue()

    while True:
        print("\n--- Interactive Queue Operations (List-based) ---")
        q.display()
        print("1. Enqueue (add request)")
        print("2. Dequeue (process request)")
        print("3. Check if Empty")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            item = input("Enter request: ")
            q.enqueue(item)
            print(f"'{item}' added to queue.")

        elif choice == '2':
            removed = q.dequeue()
            if removed:
                print(f"'{removed}' removed (processed).")
            else:
                print("Queue is empty.")

        elif choice == '3':
            if q.is_empty():
                print("Queue is empty.")
            else:
                print("Queue is not empty.")

        elif choice == '4':
            print("Exiting interactive queue operations.")
            break

        else:
            print("Invalid choice. Try again.")


# Run
interactive_queue()
```

Output:

```
...
    --- Interactive Queue Operations (List-based) ---

    Current Queue: []
    1. Enqueue (add request)
    2. Dequeue (process request)
    3. Check if Empty
    4. Exit
    Enter your choice (1-4): 2
    Queue is empty.

    --- Interactive Queue Operations (List-based) ---

    Current Queue: []
    1. Enqueue (add request)
    2. Dequeue (process request)
    3. Check if Empty
    4. Exit
    Enter your choice (1-4): 3
    Queue is empty.

    --- Interactive Queue Operations (List-based) ---

    Current Queue: []
    1. Enqueue (add request)
    2. Dequeue (process request)
    3. Check if Empty
    4. Exit
    Enter your choice (1-4): 4
    Exiting interactive queue operations.
```

Explanation:

A queue follows FIFO (First In, First Out), where the first request added is processed first.
The program uses a list with enqueue (append) and dequeue (pop(0)) operations, along with an interactive menu for user input.
It handles empty cases and continuously updates the queue, but list-based dequeue is slower (O(n)).

Task 3: Singly Linked List for Dynamic Playlist Management

Prompt:

Create a Singly Linked List in Python with insert_at_end, delete_value, and traverse methods.

Add clear comments explaining pointer updates.

Handle edge cases like empty list, single node, and deleting head.

Include sample test cases.

Code:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
```

```
        if not self.head:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def delete_value(self, key):
        if not self.head:
            print(f"'{key}' not found in playlist.")
            return

        temp = self.head

        # Delete head
        if temp.data == key:
            self.head = temp.next
            print(f"Deleted '{key}' (head).")
            return

        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next

        if temp:
            prev.next = temp.next
            print(f"Deleted '{key}'.")
        else:
            print(f"'{key}' not found in playlist.")

    def traverse(self):
        if not self.head:
            print("Playlist is empty.")
            return

        temp = self.head
        while temp:
            print(temp.data, end=" -> " if temp.next else "")
            temp = temp.next
        print()


# ----------- TEST CASES (MATCH OUTPUT STYLE) -----------

ll = LinkedList()

print("\n--- Playlist Management Test Cases ---\n")

print("Testing with empty playlist:")
ll.traverse()
ll.delete_value("Nonexistent Song")
```

```
print("\nInserting songs:")
ll.insert_at_end("Song A")
ll.insert_at_end("Song B")
ll.insert_at_end("Song C")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nDeleting 'Song B' (middle):")
ll.delete_value("Song B")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nInserting 'Song D':")
ll.insert_at_end("Song D")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nDeleting 'Song A' (head):")
ll.delete_value("Song A")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nDeleting 'Song C' (single node list):")
ll.delete_value("Song C")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nDeleting 'Song D' (last remaining):")
ll.delete_value("Song D")
ll.traverse()

print("\nAttempting to delete from an empty list:")
ll.delete_value("Any Song")
ll.traverse()

print("\nInserting into an empty list after deletions:")
ll.insert_at_end("New Song 1")
print("Current Playlist:", end=" ")
ll.traverse()

ll.insert_at_end("New Song 2")
print("Current Playlist:", end=" ")
ll.traverse()

print("\nDeleting non-existent song:")
ll.delete_value("Nonexistent Song 2")
print("Current Playlist:", end=" ")
ll.traverse()
```

Output:

```
--- Playlist Management Test Cases ---

Testing with empty playlist:
Playlist is empty.
'Nonexistent Song' not found in playlist.

Inserting songs:
Current Playlist: Song A -> Song B -> Song C

Deleting 'Song B' (middle):
Deleted 'Song B'.
Current Playlist: Song A -> Song C

Inserting 'Song D':
Current Playlist: Song A -> Song C -> Song D

Deleting 'Song A' (head):
Deleted 'Song A' (head).
Current Playlist: Song C -> Song D

Deleting 'Song C' (single node list):
Deleted 'Song C' (head).
Current Playlist: Song D

Deleting 'Song D' (last remaining):
Deleted 'Song D' (head).
Playlist is empty.

Attempting to delete from an empty list:
'Any Song' not found in playlist.
Playlist is empty.

Inserting into an empty list after deletions:
Current Playlist: New Song 1
Current Playlist: New Song 1 -> New Song 2

Deleting non-existent song:
'Nonexistent Song 2' not found in playlist.
Current Playlist: New Song 1 -> New Song 2
```

Explanation:

A singly linked list stores songs as nodes connected by pointers, allowing dynamic insertion and deletion.

Deletion updates links by skipping nodes, handling cases like head, middle, and empty list.

Traversal prints the playlist sequentially, making it suitable for ordered data like music playlists.

Task 4: Binary Search Tree for Fast Record Lookup

Prompt:

Complete a Binary Search Tree in Python with insert, search, and inorder_traversal methods.

Add docstrings and comments. Ensure correct recursive implementation.

Include test cases and explain time complexity (best vs worst case).

Code:

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

```python
class BST:
    def __init__(self):
        self.root = None

    def insert(self, value):
        def _insert(node, value):
            if node is None:
                return Node(value)
            if value < node.value:
                node.left = _insert(node.left, value)
            elif value > node.value:
                node.right = _insert(node.right, value)
            return node  # ignore duplicates

        self.root = _insert(self.root, value)

    def search(self, value):
        def _search(node, value):
            if node is None:
                return False
            if node.value == value:
                return True
            elif value < node.value:
                return _search(node.left, value)
            else:
                return _search(node.right, value)

        return _search(self.root, value)

    def inorder(self):
        result = []

        def _inorder(node):
            if node:
                _inorder(node.left)
                result.append(node.value)
                _inorder(node.right)

        _inorder(self.root)
        return result


# ----------- OUTPUT FORMAT -----------

print("\n--- Binary Search Tree for Student Records ---\n")

bst = BST()

values = [50, 30, 70, 20, 40, 60, 80, 55]
print("Inserting roll numbers:", ", ".join(map(str, values)))

for v in values:
    bst.insert(v)
```

```
print("In-order Traversal (Sorted Roll Numbers):", bst.inorder())

# Search tests
tests = [40, 80, 50, 25, 90]
for t in tests:
    print(f"Searching for roll number {t}:", bst.search(t))

# Empty BST test
print("\nTesting with an empty BST:")
empty_bst = BST()
print("Searching for 100 in empty BST:", empty_bst.search(100))
print("In-order Traversal (Sorted Roll Numbers):", empty_bst.inorder())

# Insert duplicate
print("\nInserting 50 again:")
bst.insert(50)
print("In-order Traversal (Sorted Roll Numbers):", bst.inorder())
```

Output:

```
--- Binary Search Tree for Student Records ---
ll output actions
Inserting roll numbers: 50, 30, 70, 20, 40, 60, 80, 55
In-order Traversal (Sorted Roll Numbers): [20, 30, 40, 50, 55, 60, 70, 80]
Searching for roll number 40: True
Searching for roll number 80: True
Searching for roll number 50: True
Searching for roll number 25: False
Searching for roll number 90: False

Testing with an empty BST:
Searching for 100 in empty BST: False
In-order Traversal (Sorted Roll Numbers): []

Inserting 50 again:
In-order Traversal (Sorted Roll Numbers): [20, 30, 40, 50, 55, 60, 70, 80]
```

Explanation:

A Binary Search Tree organizes data so left < root < right, enabling efficient search operations.
Insertion and search recursively reduce the search space, improving performance over linear search.
Time complexity is **O(log n)** for balanced trees and **O(n)** for skewed trees.

Task 5: Graph Traversal for Social Network Connections

Prompt:

Implement a graph using adjacency list in Python.

Add BFS and DFS traversal methods with comments.

Compare BFS and DFS and include sample test cases with clear output.

Code:

```
from collections import deque
```

```python
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u)

    def bfs(self, start):
        visited = set()
        queue = deque([start])
        result = []

        while queue:
            node = queue.popleft()
            if node not in visited:
                visited.add(node)
                result.append(node)
                queue.extend(self.graph[node])
        return result

    def dfs(self, start):
        visited = set()
        result = []

        def _dfs(node):
            if node not in visited:
                visited.add(node)
                result.append(node)
                for neighbor in self.graph[node]:
                    _dfs(neighbor)

        _dfs(start)
        return result


# -------- OUTPUT FORMAT --------

print("\n--- Social Network Connections (Graph Traversal) ---\n")

g = Graph()

edges = [
    ("A", "B"), ("A", "C"),
    ("B", "D"), ("B", "E"),
    ("C", "F"), ("E", "G")
]

print("Adding connections:")
for u, v in edges:
```

```python
    g.add_edge(u, v)
    print(f"{u} <-> {v}")

print("\nGraph Representation:")
for node in g.graph:
    print(f"{node}: {g.graph[node]}")

print("\nBFS Traversal (Nearby Connections):")
print(g.bfs("A"))

print("\nDFS Traversal (Deep Connections):")
print(g.dfs("A"))

print("\nTesting from another node (BFS from 'B'):")
print(g.bfs("B"))

print("\nTesting empty graph:")
empty = Graph()
print("BFS on empty graph:", empty.bfs("A") if "A" in empty.graph else [])
print("DFS on empty graph:", empty.dfs("A") if "A" in empty.graph else [])
```

Output:

```
...
    --- Social Network Connections (Graph Traversal) ---

    Adding connections:
    A <-> B
    A <-> C
    B <-> D
    B <-> E
    C <-> F
    E <-> G

    Graph Representation:
    A: ['B', 'C']
    B: ['A', 'D', 'E']
    C: ['A', 'F']
    D: ['B']
    E: ['B', 'G']
    F: ['C']
    G: ['E']

    BFS Traversal (Nearby Connections):
    ['A', 'B', 'C', 'D', 'E', 'F', 'G']

    DFS Traversal (Deep Connections):
    ['A', 'B', 'D', 'E', 'G', 'C', 'F']

    Testing from another node (BFS from 'B'):
    ['B', 'A', 'D', 'E', 'C', 'G', 'F']

    Testing empty graph:
    BFS on empty graph: []
    DFS on empty graph: []
```

Explanation:

A graph represents users and their connections using an adjacency list structure.
BFS explores nodes level-by-level using a queue, while DFS explores deeply using recursion.
BFS is ideal for shortest paths, whereas DFS is useful for exploring full connection paths.